

Reviewer Pack — Minimal p-adic Order of Local Zeta Functions and SAT Clause ...

Entry 3510b1fe5d63 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 21:38:19 UTC

Minimal p-adic Order of Local Zeta Functions and SAT Clause Depth

Entry ID: 3510b1fe5d63 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 21:38:19 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Local Zeta Function Theory) **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

For every CNF formula φ with n variables, the minimal p-adic order of its local zeta function is $O(n^{(1/2)})$, i.e., $\text{ord}_p(Z(\varphi)) = \Theta(n^{(1/2)})$.

Rationale (proposer's reasoning):

Local zeta functions in p-adic analysis provide a way to study arithmetic properties of complex systems. A connection between their order and clause depth could reveal a deeper arithmetic structure underlying SAT solvability.

Taxonomy category: Arithmetic_Properties_of_Boolean_Formulas (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 331fa0dfc4433424

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the median ratio of clause depth to p-adic order for at least 95% of CNF formulas with n variables is less than or equal to 4, and no seed produces a ratio greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic analysis AND local zeta function AND SAT clause complexity - minimal p-adic order AND local zeta function AND Boolean satisfiability - Clause depth in SAT AND p-adic order of Zeta functions

Top relevant hits considered: - [http://arxiv.org/abs/2201.08874v2] Note on p -adic Local Functional Equation - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p -adic Equiangular Lines and p -adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/hep-th/9410058v3] p -Adic description of Higgs mechanism I: p -Adic square root and p -adic light cone - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/2406.13557v1] satsuma: Structure-based Symmetry Breaking in SAT - [http://arxiv.org/abs/1105.1436v1] Solving Rubik's Cube Using SAT Solvers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.randint(-1, 0) * (i + 1) for i in range(n)]
            if all(c == 0 for c in clause):
                continue
            clauses.append(clause)
        return clauses

    def dpll_solve(clauses, assignment=[]):
        if not clauses:
            return True
        var = next((v for v in range(1, len(clauses) + 1) if v not in [abs(c) for c in assignment]))
        if var is None:
            return False

        def propagate(var, value):
            new_clauses = []
            for clause in clauses:
                if any(abs(v) == var and c != value * v for c in clause):
                    continue

        def dpll_solve(clauses, assignment=[]):
            if not clauses:
                return True
            var = next((v for v in range(1, len(clauses) + 1) if v not in [abs(c) for c in assignment]))
            if var is None:
                return False

            def propagate(var, value):
                new_clauses = []
                for clause in clauses:
                    if any(abs(v) == var and c != value * v for c in clause):
                        continue
```

```

        elif all(abs(v) != abs(c) for c in clause):
            return None
        new_clause = [c for c in clause if abs(c) != var]
        if not new_clause:
            return None
        new_clauses.append(new_clause)
    return new_clauses

if dpll_solve(propagate(var, 1), assignment + [var]):
    return True
if dpll_solve(propagate(-var, -1), assignment + [-var]):
    return True
return False

def p_adic_order(clause):
    # Simplified approximation for demonstration purposes
    return len(clause) ** (1/2)

n = 5
trials = 30
total_ratio = 0
max_n = n

for _ in range(trials):
    cnf = generate_cnf(n)
    if not dpll_solve(cnf):
        continue

    clause_depth = len(cnf)
    p_order = p_adic_order(cnf[0])

    if p_order == 0:
        continue

    ratio = clause_depth / p_order
    total_ratio += ratio
    max_n = max(max_n, n)

if trials == 0:
    return {
        "metric_name": "ratio",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": max_n,
        "conjecture_holds": False,
        "counterexample": "no_valid_cnf_found"
    }

mean_ratio = total_ratio / trials
support_fraction = (mean_ratio <= 4)

return {
    "metric_name": "ratio",
    "metric_value": mean_ratio,
    "instances_tested": trials,
    "n_max": max_n,

```

```

        "conjecture_holds": support_fraction,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_ratio = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    elif any(r['metric_value'] > 10 for r in results):
        first_failing_seed = next((r['seed'] for r in results if r['metric_value'] > 10), None)
        print(f"RESULT: FALSIFIED counterexample=\"ratio_exceeds_10\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d350daf4.py", line 107, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d350daf4.py", line 67, in run_trial
    if not dpll_solve(cnf):
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d350daf4.py", line 50, in dpll_solve
    if dpll_solve(propagate(var, 1), assignment + [var]):
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d350daf4.py", line 40, in propagate
    if any(abs(v) == var and c != value * v for c in clause):
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d350daf4.py", line 40, in <genexpr>
    if any(abs(v) == var and c != value * v for c in clause):
           ^
NameError: name 'v' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the median ratio of clause depth to p-adic order for at least 95% of CNF formulas with n variables as required by the support condition. | next: Re-run the test without errors to collect the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15141
2	propose	ollama_remote	glm4:latest	0	0	12836
3	preregistration	ollama_remote	glm4:latest	0	0	9583
4	novelty	ollama_remote	glm4:latest	0	0	8682
5	novelty	ollama_remote	glm4:latest	0	0	10141
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14567
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8819
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10911
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11493
10	judge	ollama_remote	glm4:latest	0	0	15746

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117919 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/3510b1fe5d63.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3510b1fe5d63.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3510b1fe5d63.tar.gz (if generated)